MODULE 27 · DOMAIN 4 - CLOUD NATIVE ARCHITECTURE

Cloud native principles and autoscaling

What makes a system cloud native, resilience patterns, the four autoscalers, serverless, and cost.

DOMAIN 4 of 4 EXAM WEIGHT 12% MODULE 27 of 35

admin@sthithapragna.com

Cloud native systems are **loosely coupled**, **resilient** and **automated**, so they can change often and fail safely.

The CNCF's definition, in plain words

- **Build and run scalable applications** in dynamic environments: public, private and hybrid clouds.
- **Typical techniques:** containers, service meshes, microservices, immutable infrastructure, declarative APIs.
- **The result:** loosely coupled systems that are resilient, manageable and observable.
- **With strong automation,** engineers make big changes often and predictably, with little toil.

> ON THE EXAM
>
> Cloud native is about how you build and run, not where. It applies on-premises too.

Monolith versus microservices

MONOLITH

One deployable

- Simple to build, test and run at first
- One release for everyone
- Scales only as a whole
- One bug can take everything down

MICROSERVICES

Many small services

- Each deployed and scaled independently
- Teams own and release their service
- Network calls, versioning, tracing needed
- Failures can be contained

THE TIEBREAKER

Microservices trade code complexity for operational complexity. Kubernetes exists to handle the latter.

Traits of a cloud native application

TRAIT	IN PRACTICE
Immutable	Replace, never patch in place: new image, new Pods
Declarative	Describe the desired state; controllers make it so
Disposable	Start fast, stop gracefully on SIGTERM
Stateless where possible	State lives in databases and stores, not in Pods
Observable	Metrics, logs and traces by design
Configured from outside	The same image in every environment

ON THE EXAM

These echo the twelve-factor app: config in the environment, disposable processes, logs as streams.

Resilience patterns

- **Redundancy:** several replicas, spread across nodes and zones (module 12).
- **Timeouts and retries with back-off,** so one slow call does not stall everything.
- **Circuit breakers:** stop calling a failing service for a while; fail fast instead.
- **Graceful degradation:** show cached timetables when live fares are unavailable.
- **Chaos engineering:** break things on purpose to prove resilience. LitmusChaos and Chaos Mesh are CNCF projects.

> ON THE EXAM
>
> Resilience is designed in. Kubernetes restarts containers; it cannot make your calls time out sensibly.

Kubernetes scales in three directions: more Pods, bigger Pods, and more nodes.

The four autoscalers

AUTOSCALER	CHANGES	TRIGGERED BY
`HorizontalPodAutoscaler (HPA)`	The number of Pods	CPU, memory or custom metrics per Pod
`VerticalPodAutoscaler (VPA)`	Each Pod's requests and limits	Observed usage over time
`Cluster Autoscaler, Karpenter`	The number of nodes	Pods Pending for lack of room
`KEDA`	The number of Pods, including zero	Events: queue length, lag, schedules

ON THE EXAM

HPA is built in. VPA, the node autoscalers and KEDA are add-ons.

How the HPA decides

01
Measure
Reads metrics, e.g. CPU from metrics-server.

02
Compare
Average usage against the target, say 70%.

03
Calculate
desired = current x (actual / target), rounded up.

04
Scale
Sets the Deployment's replicas, within min and max.

05
Stabilise
Waits 5 minutes before scaling down, to avoid flapping.

ON THE EXAM

Four Pods at 140% of a 70% target: 4 x 140/70 = 8 Pods.

An HPA for the booking API

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: booking-api, namespace: quayside}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: booking-api}
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 70}}
```

5 Scales the booking API Deployment.

6-7 Never fewer than 3, never more than 20.

10 70% of each Pod's requested CPU. Pods must declare requests.

HPA details the exam likes

- **Utilisation is a percentage of requests.** No requests: the target shows unknown.
- **Custom and external metrics** come through adapters, such as the Prometheus Adapter, or KEDA.
- **Scale-down waits five minutes** by default: the stabilisation window.
- **The HPA owns replicas.** Hand edits, or a replica count in Git, fight it.

> ON THE EXAM
>
> With GitOps, leave replicas out of Git for HPA-managed workloads, or the two will argue.

HPA versus VPA

HPA

More Pods

- Adds or removes identical Pods
- For stateless apps that scale out
- Reacts to load quickly
- Built into Kubernetes

VPA

Right-sized Pods

- Sets requests and limits from real usage
- For apps that cannot scale out
- Recommends, or applies automatically
- Can use in-place resize (module 12)

THE TIEBREAKER

Do not let HPA and VPA both act on the same CPU or memory metric; they fight.

Scaling Pods versus scaling nodes

HPA, KEDA

More Pods

- Triggered by load on the app
- Needs room on existing nodes
- Fails to help if nodes are full
- New Pods stay Pending

CLUSTER AUTOSCALER, KARPENTER

More nodes

- Triggered by Pending Pods
- Adds nodes that fit them
- Removes nodes that sit underused
- Karpenter picks node sizes itself

THE TIEBREAKER

Pods overloaded: HPA. Pods Pending with Insufficient cpu: a node autoscaler. They work together.

KEDA: event-driven autoscaling

- **A graduated CNCF project** since 2023: Kubernetes Event-Driven Autoscaling.
- **Scales on events:** queue length, Kafka lag, a Prometheus query, a schedule.
- **Scales to zero** when idle, and back up when work arrives.
- **A ScaledObject** describes the trigger; KEDA manages an HPA underneath.

> ON THE EXAM
>
> 'Scale to zero, and scale on queue length': KEDA.

Serverless: scale to zero, pay only while code runs, and let the platform handle the rest.

Serverless on Kubernetes

- **Knative**, CNCF graduated in October 2025: serverless and event-driven apps on Kubernetes.
- **Knative Serving:** scale to zero, request-based autoscaling, revisions, traffic splitting.
- **Knative Eventing:** connects event sources to services.
- **CloudEvents,** a CNCF specification, describes event data the same way everywhere.
- **The trade-off: cold starts.** The first request after idle waits for a Pod to start.

> ON THE EXAM
>
> 'Scale HTTP services to zero on Kubernetes': Knative. 'A common format for events': CloudEvents.

Cost: FinOps for Kubernetes

- **See the cost:** by namespace, team and label. OpenCost is a CNCF project for this.
- **Right-size requests:** over-requesting reserves capacity nobody uses (module 12).
- **Scale down, even to zero,** outside busy hours.
- **Spot or pre-emptible nodes** for work that tolerates interruption.

> ON THE EXAM
>
> In Kubernetes, you pay for what you request, not what you use. Requests drive cost.

Six architecture decisions. For scaling, ask: **what is short**, Pods, size, nodes, or none at all?

> The stateless booking API sees five times more traffic every evening. CPU per Pod climbs sharply at the peak.
>
> Quayside wants more Pods automatically when load rises, and fewer again afterwards.

Which autoscaler fits?

A A VerticalPodAutoscaler

B A Cluster Autoscaler

C A HorizontalPodAutoscaler on CPU

D A CronJob that scales at fixed times

◆ PAUSE AND DECIDE

Which autoscaler fits?

A A VerticalPodAutoscaler

Makes Pods bigger, not more numerous.

B A Cluster Autoscaler

Adds nodes, not Pods.

C A HorizontalPodAutoscaler on CPU

More identical Pods as CPU rises; fewer after.

D A CronJob that scales at fixed times

Fixed times miss unexpected peaks.

THE SIGNAL

Stateless app, load-driven, more Pods: HPA.

> An HPA targets 70% CPU for the web front end. kubectl get hpa shows the target as unknown/70%, and it never scales.
>
> metrics-server is running and kubectl top pods works. The web containers set no resource requests.

What is the fix?

A Add CPU requests to the web containers

B Install Grafana to supply the metrics

C Raise the HPA's maxReplicas to 50

D Switch the Deployment strategy to Recreate

◆ PAUSE AND DECIDE

What is the fix?

A Add CPU requests to the web containers

Utilisation is a share of requests; without them there is no ratio.

B Install Grafana to supply the metrics

Grafana displays metrics; it supplies nothing to the HPA.

C Raise the HPA's maxReplicas to 50

The HPA cannot scale at all yet; the cap is irrelevant.

D Switch the Deployment strategy to Recreate

Rollout strategy has nothing to do with scaling.

THE SIGNAL

'unknown' target with metrics-server working: missing requests.

> During a sale, the booking API's HPA asks for 12 replicas. Eight run; four stay Pending with: Insufficient cpu.
>
> Every node is already full of other workloads.

What should add the missing capacity?

A Raise the HPA's maxReplicas

B Add a VerticalPodAutoscaler

C Set the Pods' CPU requests to zero

D A Cluster Autoscaler or Karpenter

◆ PAUSE AND DECIDE

What should add the missing capacity?

A Raise the HPA's maxReplicas

The HPA already asked for 12; the nodes are the limit.

B Add a VerticalPodAutoscaler

Bigger Pods fit even worse on full nodes.

C Set the Pods' CPU requests to zero

Hides the shortage and overloads the nodes.

D A Cluster Autoscaler or Karpenter

Pending Pods trigger new nodes that fit them.

THE SIGNAL

Pods Pending with Insufficient cpu: a node autoscaler.

> The payment worker reads jobs from a message queue. At night the queue is empty for hours; at peak it holds thousands of jobs.
>
> Quayside wants zero workers when the queue is empty, and more workers as it grows.

Which tool fits?

A An HPA on the workers' CPU

B KEDA, scaling on queue length

C A VerticalPodAutoscaler

D A DaemonSet, one worker per node

◆ PAUSE AND DECIDE

Which tool fits?

A. An HPA on the workers' CPU
Cannot reach zero on its own, and CPU lags the queue.

B. KEDA, scaling on queue length
Scales on the queue itself, down to zero.

C. A VerticalPodAutoscaler
Resizes Pods; never removes them.

D. A DaemonSet, one worker per node
Ties workers to nodes, not to the queue.

THE SIGNAL

'Scale to zero on queue length': KEDA.

> Quayside's legacy reporting service can only ever run as a single instance.
>
> Nobody knows its right size. It is OOMKilled at month end, and wastes memory the rest of the time.

Which autoscaler helps most?

A. A HorizontalPodAutoscaler

B. KEDA with a schedule trigger

C. A Cluster Autoscaler

D. A VerticalPodAutoscaler

◆ PAUSE AND DECIDE

Which autoscaler helps most?

A. A HorizontalPodAutoscaler
Adds instances, which this service cannot run.

B. KEDA with a schedule trigger
Scales instance count, which is fixed at one.

C. A Cluster Autoscaler
Adds nodes; the Pod's own size is the problem.

D. A VerticalPodAutoscaler
Learns real usage and right-sizes requests and limits.

THE SIGNAL

Cannot scale out, size unknown: VPA.

> Quayside's booking and web features live in one monolith, released weekly after every team coordinates.
>
> Teams want to release independently, and to scale booking separately from the website during sales.

Which direction fits these goals?

A Split into microservices, deployed independently

B Run the monolith on much larger servers

C Put every component in one multi-container Pod

D Run the monolith as a StatefulSet

◆ PAUSE AND DECIDE

Which direction fits these goals?

A Split into microservices, deployed independently

Independent release and scaling, at the cost of more operational work.

B Run the monolith on much larger servers

Scales everything together; releases stay coupled.

C Put every component in one multi-container Pod

One Pod scales as one unit, like the monolith.

D Run the monolith as a StatefulSet

Adds identity, not independence.

THE SIGNAL

Independent releases and independent scaling: microservices.

Module 27 in five lines

- Cloud native: **loosely coupled, resilient, observable, automated**. How, not where.
- **HPA** more Pods (needs requests); **VPA** bigger Pods; don't combine them on the same metric.
- **Cluster Autoscaler / Karpenter** add nodes for Pending Pods.
- **KEDA** scales on events, to zero. **Knative** (graduated 2025) for serverless.
- **You pay for requests.** Right-size, scale down, use spot for tolerant work.

Last teaching module: **the community behind all of it**, and how cloud native projects are governed.